

Mais itens

Dois itens novos para escrever com as próprias mãos:
uma Poção de força e um Anel de proteção.

Depois da explicação de hoje.

Este documento não repete o código já mostrado. Ele diz o que construir — quem escreve é você.

ANTES DE COMEÇAR

Onde você está

Você acabou de ver, na explicação, como a `PocaoDeVida` e a `Bomba` viraram `PocaoDeForca` e `AneLDeProtecao`. Agora é a sua vez de escrever as duas, sozinho, no seu próprio `itens.py`.

Confira que você tem isto antes de seguir:

ARQUIVO	O QUE PRECISA TER
<code>personagem.py</code>	Personagem com <code>ataque</code> e <code>defesa</code> como atributos comuns.
<code>itens.py</code>	<code>Item</code> , <code>PocaoDeVida</code> , <code>Bomba</code> e <code>Inventario</code> .
<code>main.py</code>	A classe <code>Fase</code> e a lista <code>fases</code> já funcionando.

SE O SEU JOGO NÃO ESTIVER NESSE PONTO

Não perca a aula de hoje consertando a de ontem. Pegue os quatro arquivos prontos em `Codigo Base/`, na mesma pasta deste PDF, e comece os exercícios a partir dali.

Nenhum arquivo novo entra hoje. Os dois itens são classes novas dentro do `itens.py` que já existe, do lado da `PocaoDeVida` e da `Bomba`.

EXERCÍCIO 1

Poção de força

Crie a classe `PocaoDeForca` no `itens.py`. Ela segue exatamente o mesmo molde da `PocaoDeVida` — só o que ela guarda e o que ela faz em `usar` muda.

O QUE ELA PRECISA TER

- Herdar de `Item`.
- `__init__(self)` chamando `super().__init__("Poção de força")`, e guardando `self.forca = 5`.
- `usar(self, dono, inimigo)` que aumenta o `ataque` de `dono` em `self.forca`, e imprime uma mensagem avisando.

Se travar em alguma linha, olhe a `PocaoDeVida` ao lado da sua tela. A diferença entre as duas é pequena.

CHECKPOINT 1

A classe `PocaoDeForca` existe no `itens.py`.

`usar` aumenta o `ataque` de quem usou, e não mexe em mais nada.

TESTE DO CHECKPOINT 1

```
h = Guerreiro("Thoric")
print("antes:", h.ataque)
PocaoDeForca().usar(h, None)
print("depois:", h.ataque)
```

Resultado esperado:

```
antes: 18
depois: 23
```

Se o número não mudar: confira se `usar` está escrevendo `dono.ataque`, e não `self.ataque`.

EXERCÍCIO 2

Anel de proteção

Agora crie a classe `AnelDeProtecao`, do mesmo jeito — mas pensando em `defesa` em vez de `ataque`. Antes de escrever, responda sozinho: quantas linhas da `PocaoDeForca` precisam mudar?

O QUE ELE PRECISA TER

- Herdar de `Item`.
- `__init__(self)` chamando `super().__init__("Anel de proteção")`, e guardando `self.protecao = 5`.
- `usar(self, dono, inimigo)` que aumenta a `defesa` de `dono` em `self.protecao`, e imprime uma mensagem avisando.

CHECKPOINT 2

A classe `AnelDeProtecao` existe no `itens.py`.

`usar` aumenta a `defesa` de quem usou — não o `ataque`. Esse é o erro mais fácil de copiar e colar sem perceber.

TESTE DO CHECKPOINT 2

```
h = Guerreiro("Thoric")
print("antes:", h.defesa)
AnelDeProtecao().usar(h, None)
print("depois:", h.defesa)
```

Resultado esperado:

```
antes: 8
depois: 13
```

Se o ataque mudou em vez da defesa: sobrou uma linha copiada da `PocaoDeForca` sem trocar o atributo.

EXERCÍCIO 3

Colocar nas fases

Os dois itens só valem alguma coisa se o jogo realmente entregar eles. No `main.py`, troque o `premio` de pelo menos duas fases da lista `fases` para usar os itens novos.

```
Fase("Trilha da Montanha", "Um Orc bloqueia o caminho.",  
    Orc(), PocaoDeForca()),  
Fase("Ponte Quebrada", "Um Troll ronca embaixo da ponte.",  
    Troll(), AnelDeProtecao()),
```

CHECKPOINT 3, O ÚLTIMO

Pelo menos uma fase dá `PocaoDeForca` de prêmio, e pelo menos outra dá `AnelDeProtecao`.

O jogo roda do início ao fim sem erro, e a mochila mostra os itens novos depois de ganhá-los.

TESTE FINAL

Jogue até vencer a fase que dá `PocaoDeForca`. Na tela seguinte, escolha **3 - Abrir a mochila**.

Resultado esperado: a Poção de força aparece na lista, com o nome certo.

Antes de encerrar, confira

- ☐ `PocaoDeForca` e `AnelDeProtecao` existem em `itens.py`
- ☐ Nenhuma das duas herda de `Personagem` — as duas herdam de `Item`
- ☐ Cada uma mexe em só um atributo do dono: ataque ou defesa, nunca os dois
- ☐ Pelo menos duas fases usam os itens novos como prêmio
- ☐ O jogo roda do início ao fim sem erro na tela